Group 5 – Library Management System: Test Case Assignment

Oct 21, 2026 · @Shakthi

1. Assignment details

This group designs and documents six or more test cases for a college Library Management System, covering issuing, borrowing limits, availability, returns and transaction records.

Field	Detail
Course	Software Engineering
Assignment	In-class group assignment, Units I–IV: Software Engineering Concepts and Test Case Design
Group	Group 5 (of 10)
Application	Library Management System
Related topics	Functional testing, data validation, state changes
Duration	2 hours
Submission	One assignment per group, as a soft copy
Institution	SSMRV College, Bengaluru (BCA, Bengaluru City University)

Team member	Roll no.	Contribution
Shakthi		

2. Rules, guidelines and process

Eight rules come from the assignment sheet; the guidelines and seven-step process below turn them into a repeatable way of working.

Rules (from the assignment sheet)

1. Answer only the question assigned to the group (Group 5).
2. Understand the system requirements before writing any test case.
3. Prepare at least six test cases.
4. Include positive, negative and boundary or special-condition test cases wherever applicable.
5. Write a clear, specific expected result for every test case.
6. Never assume a test case passed or failed without running it. On paper, write "Not Executed" under Actual Result and Status.
7. Every group member must contribute.
8. Be ready to present the work to the class.

Guidelines for writing good test cases

- **One objective per test case.** If a case checks two things, split it, so a failure points to one cause.
- **State the starting condition.** Write the precondition (for example, "Student has 2 books issued") at the start of the Test Input / Action cell.
- **Use exact data.** Name the student, book and count ("Student S101 issues Book B05"), not "a student issues a book".
- **Make expected results observable.** Describe what the system shows or stores, such as a message, a status change or a record, not what it "should" do in general.
- **Keep IDs sequential.** Use TC-01, TC-02 and so on, and keep the same order in Parts B and C.
- **Cover the three kinds.** Positive (valid use), negative (invalid use) and boundary (at the limit, such as the third and fourth book).

The process, step by step

1. **Read the requirements and mark the rules.** Highlight each testable statement (limit of three, only available books, returns restore availability, no double return, records kept) and label them R1–R5. *Why:* a test case is only meaningful if it traces back to a

requirement.

2. **Complete Part A.** Write the purpose, users, four functional and two non-functional requirements. *Why:* this proves the group understands the system before testing it, which is rule 2.
3. **Pick test techniques.** Use equivalence partitioning (valid and invalid inputs), boundary value analysis (the 3-book limit) and state transition (a book moving Available → Issued → Available). *Why:* techniques give systematic coverage instead of guesses.
4. **List test conditions.** For each requirement write what must be true and what must be rejected. *Why:* each condition becomes one or more test cases, so none is forgotten.
5. **Write the test cases in the required table.** Fill ID, Objective, Input/Action and Expected Result, and write "Not Executed" in Actual Result and Status. *Why:* this is the graded deliverable and the format must match exactly.
6. **Check coverage and review.** Confirm each of R1–R5 has at least one test case, and that there are at least two positive, two negative and one boundary case. Another member reviews it. *Why:* a second reader catches vague wording and gaps.
7. **Compile, divide the presentation and submit.** Complete Part C, record each member's contribution, export the soft copy and assign who presents which part. *Why:* rules 7 and 8 require shared contribution and a class presentation.

3. Part A: Understanding requirements

The Library Management System lets a college library issue books to students, take them back, and keep an accurate record of every transaction.

1. Purpose of the application

The application manages book issues and returns in a college library. It enforces the borrowing rules automatically, keeps the availability of each book correct, and stores a history of issued and returned books.

2. Users of the application

- **Student:** borrows and returns books.
- **Librarian:** issues and receives books, and checks records. This is the main operator of the system.
- **Administrator:** manages the catalogue and student accounts, and reviews transaction records.

3. Four functional requirements

ID	Functional requirement
R1	A student may borrow a maximum of three books at a time.
R2	Only books that are currently available can be issued.
R3	A returned book becomes available again for other students.
R4	The system must prevent the same issue from being returned twice.

The system must also maintain records of issued and returned books (R5), which the test cases in Part B cover as well.

4. Two non-functional requirements

ID	Non-functional requirement
NFR1	**Performance:** issuing or returning a book completes and shows its result within 3 seconds under normal load.
NFR2	**Data integrity and security:** transaction records are stored reliably, cannot be edited by students, and stay consistent if an operation fails midway.

4. Part B: Test case design

Eight test cases cover every requirement. The data (S101 for a student, B01 for a book) is sample data the tester sets up first; "Pre" is the starting condition and "Action" is what the tester does. Nothing has been run, so Actual Result and Status are "Not Executed" for all.

Test Case ID	Objective	Test Input / Action	Expected Result	Actual Result	Status
TC-01	Issue an available book to a student (positive)	Pre: Student S101 has 0 books issued; Book B01 is Available. Action: issue B01 to S101.	Issue succeeds with a confirmation message. B01 becomes Issued, S101′s count becomes 1, and an issue record (student, book, date) is created.	Not Executed	Not Executed
TC-02	Reject issuing a book that is already issued (negative)	Pre: B02 is Issued to student S102. Action: issue B02 to S101.	Issue is rejected with a "book not available" message. B02 stays Issued to S102, S101′s count is unchanged, and no new record is created.	Not Executed	Not Executed
TC-03	Allow the third book, the borrowing limit (boundary)	Pre: Student S103 has 2 books issued; B05 is Available. Action: issue B05 to S103.	Issue succeeds. S103′s count becomes 3, B05 becomes Issued, and a record is created.	Not Executed	Not Executed
TC-04	Reject a fourth book, one over the limit (negative, boundary)	Pre: S103 has 3 books issued; B06 is Available. Action: issue B06 to S103.	Issue is rejected with a "borrowing limit of 3 reached" message. B06 stays Available, S103′s count stays 3, and no record is created.	Not Executed	Not Executed
TC-05	Return an issued book (positive, state change)	Pre: B01 is Issued to S101. Action: return B01 for S101.	Return succeeds. B01 becomes Available, S101′s count drops by 1, and a return record with the return date is created.	Not Executed	Not Executed
TC-06	Prevent returning the same book twice (negative)	Pre: B01 was already returned (TC-05) and is Available. Action: try to return B01 for S101 again.	Return is rejected with an "already returned" message. B01 stays Available, S101′s count does not change, and no duplicate return record is created.	Not Executed	Not Executed

Test Case ID	Objective	Test Input / Action	Expected Result	Actual Result	Status
TC-07	Reissue a returned book to another student (positive, state change)	Pre: B01 is Available after TC-05; student S104 has 0 books. Action: issue B01 to S104.	Issue succeeds. B01 becomes Issued to S104 and a new issue record is created for S104.	Not Executed	Not Executed
TC-08	Verify transaction records are complete and accurate (positive)	Pre: TC-01 and TC-05 are done for S101 and B01. Action: open the transaction history for S101.	History shows one issue entry and one return entry for B01 with dates, in order. Rejected attempts (TC-02, TC-04, TC-06) do not appear as transactions.	Not Executed	Not Executed

Requirement coverage

Requirement	Covered by	Test type
R1: maximum of three books	TC-03, TC-04	Boundary, negative
R2: only available books issued	TC-01, TC-02	Positive, negative
R3: returned books become available	TC-05, TC-07	Positive, state change
R4: no double return	TC-06	Negative
R5: records of issues and returns	TC-01, TC-05, TC-08	Positive

5. Part C: Test case analysis

1. Two positive test cases

- **TC-01** checks that an available book is issued to a student with no books. It proves the core issue flow works with valid data.
- **TC-05** checks that an issued book can be returned and becomes Available. It proves the return flow and the state change work.

2. Two negative test cases

- **TC-02** tries to issue a book that is already issued. The system must refuse it and leave the existing issue untouched.
- **TC-06** tries to return the same book twice. The system must refuse the second return and create no duplicate record.

3. One boundary test case

TC-03 issues the third book to a student who holds two, which is exactly at the limit of three. Bugs cluster at limits, so this case has its partner TC-04, which issues the fourth book and must be rejected.

4. One possible software defect

An off-by-one error in the borrowing limit check: if the code tests "books issued <= 3" instead of "< 3", a student could borrow a fourth book. TC-04 would expose it, because the expected result is a rejection and the system would issue the book instead. A related defect is a double return that reduces a student's count twice, which TC-06 would expose.

5. How these test cases improve software quality

The test cases turn each requirement into a check that can be repeated, so a requirement is never left untested. Negative and boundary cases catch the errors users actually trigger, such as unavailable books, over-limit borrowing and duplicate returns. Defects found on paper or in testing cost far less to fix than defects found by students at the library desk. Clear expected results also let any tester judge pass or fail the same way, and the set can be re-run after every change to catch regressions. Together they raise confidence that availability, limits and records stay correct.

6. Presentation plan and checklist

Each member presents one part, so all four sections are covered and every member contributes.

Part	Presenter	Time
Part A: requirements	Member 1	2 min
Part B: TC-01 to TC-04	Member 2	3 min

Part	Presenter	Time
Part B: TC-05 to TC-08	Member 3	3 min
Part C: analysis and conclusion	Member 4	2 min

- [ ] Every test case has an objective, specific input and a clear expected result
- [ ] Actual Result and Status say "Not Executed" on all rows
- [ ] At least two positive, two negative and one boundary case are identified
- [ ] Team contributions are filled in on page 1
- [ ] Soft copy exported and submitted by the deadline